

Electrothon'26 - Signal Processing Module

Gowtham Krishna Katakam

July 2026

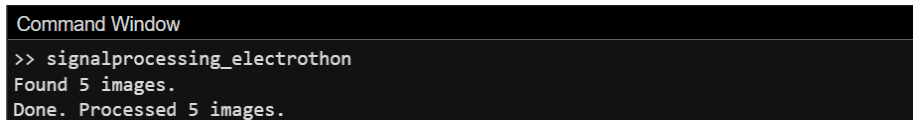
In this report, we will go over my MATLAB code for the Signal Processing Module in Electrothon '26.

1 Basics

The attached .m script first creates a DataStore of the images to be processed by navigating to a set file path (in this case, a folder in my Downloads).

It also checks for (and creates, if not found) an output folder using another fixed path (again, another folder in my Downloads).

If the images are found and the DataStore is successfully created, the script then begins running my image processing pipeline on each image in the DataStore. Apart from returning the processed image, it also returns a diagnostic figure for each image - for bug-fixing and visualization.



```
Command Window
>> signalprocessing_electrothon
Found 5 images.
Done. Processed 5 images.
```

Figure 1: Expected Output with 5 images in input folder

The major steps in the processing pipeline are:

1. Pre-processing
2. Spatial Median Filter
3. Spectral Adaptive Notch Filter
4. Post-processing

2 Pre-processing

This part of the pipeline is very simple, it prepares the image for the rest of the pipeline simply by converting the image to grayscale (i.e. binarizing it) and then making it double precision. As all the images provided are already binarized, there is no change to the given images itself. The image shown below (and all other images displayed henceforth are of the 5th image provided)

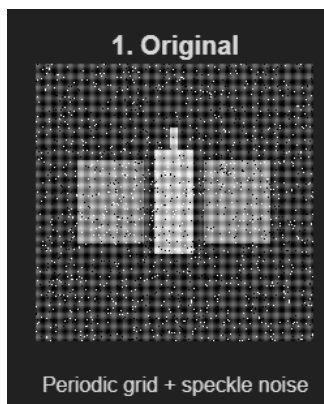


Figure 2: Image after Pre-Processing

3 Spatial Median Filter

This step applies a median filter of size 3×3 , using the inbuilt function `medfilt2`. This cleanly removes the pixelated salt and pepper noise pattern present on the images.

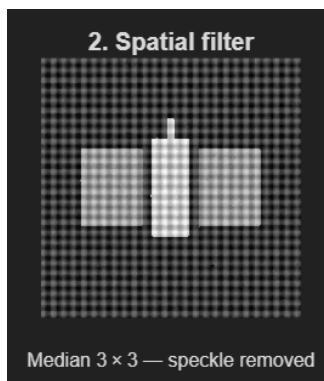


Figure 3: Image after Spatial Filtering

4 Spectral Adaptive Notch Filtering

This step is the most math-heavy step of the pipeline, as it is adaptive. The overall aim is to locate the noise frequencies by converting the image into the frequency domain and then eliminating them.

First, we convert the image using a 2D fast Fourier transform - the MATLAB function `fft2` (we use `fftshift` simply for ease of visualization). We then apply logarithmic compression to the magnitude of the Fourier image, for better scaling.

The first problem we will encounter is that we cannot directly scan for frequency peaks as that would also return the frequencies around zero (zero-frequency = DC). Thus, we create a mesh centered around the zero-frequency with a set radius ($0.04 * \text{rows}$ was what was found to be optimal after testing with various values, and $\text{rows} = \text{minimum between the number of rows and the number of columns}$).

Now, with the DC protective zone created, we can find the peak frequencies outside the zone using the quantile function. Setting the second parameter to 0.999 makes sure our threshold is only below the poles formed by the spectral noise in the image.

Finally, we create a notch mask (0 = remove, 1 = keep) and set everything above the threshold to 0 and everything below to 1 (including the DC safe zone). We apply the inverse FFT shift, the inverse 2-D FFT, and convert the image to the real domain. Another problem that arose was the fact that the image was no longer between 0 to 1 as expected for a grayscale and that is rectified by normalizing the image.

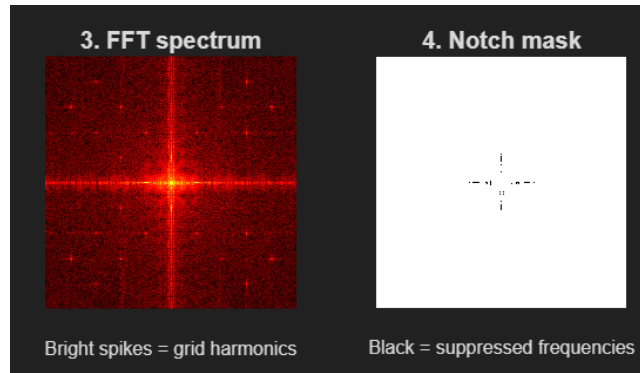


Figure 4: FFT Spectrum on left and Notch Mask on right

5 Post-Processing

To reduce the sharpening and contrast of the image, we clip the bottom and top 1%, making the image slightly smoother and mostly similar to the original image.

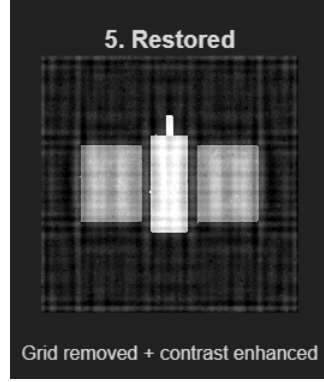


Figure 5: Image after Pre-Processing

All the above images are part of the diagnostics provided by the script for each image processed in the pipeline. Shown previously were the diagnostics for image 5, the others are displayed as shown.

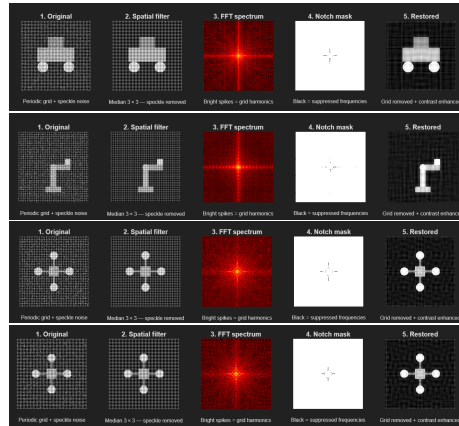


Figure 6: Image after Pre-Processing